

Rendu individuel

Anis Douadi

Orchestration Kubernetes / Helm & on-premise / PRA

Auteur	Anis Douadi
Rôle	Orchestration Kubernetes / Helm & on-premise / PRA
Classe / Promo	M2 DO C — 2025-2026
Période	Janvier – Juin 2026
Projet	Plateforme de gestion d'un cabinet d'audioprothèse
Domaine	Chart Helm, autoscaling, site on-premise, MinIO chiffré, Ansible

SUP DE VINCI — Expert en Systèmes d'Information — Mastère DevOps — Classe **M2 DO C** — Promotion 2025-2026. Document réalisé dans le cadre du projet d'étude de fin d'année.

1. Contexte et rôle dans le projet

J'ai été responsable de l'orchestration des conteneurs sur Kubernetes et du volet hybride du projet, c'est-à-dire le site on-premise et le plan de reprise d'activité. Ces sujets touchent directement à la résilience et à la pérennité des données : ce sont eux qui déterminent la capacité de la plateforme à survivre à une panne et à protéger durablement les informations du cabinet.

Mon travail a consisté à transformer les images produites par l'équipe en un déploiement Kubernetes cohérent, résilient et paramétrable, puis à concevoir la chaîne de sauvegarde reliant le cloud à un site on-premise simulé. J'ai donc collaboré étroitement avec Zaafer sur l'application, Elyess sur l'intégration du déploiement dans la chaîne, et Adame sur la sécurité et la supervision du cluster.

Le cahier des charges décrivant explicitement une architecture hybride mêlant cloud public et serveurs on-premise pour les données sensibles, mon rôle était aussi de donner corps à cette vision, en démontrant concrètement comment des sauvegardes chiffrées peuvent être répliquées hors du cloud, sur un site maîtrisé par l'organisation.

2. Ma contribution détaillée

2.1 Conception du chart Helm

J'ai conçu le chart Helm qui décrit l'ensemble des objets Kubernetes de l'application : les déploiements du backend et du frontend, les services qui les exposent, l'Ingress qui route le trafic entrant, le secret contenant la chaîne de connexion, l'autoscaling et la définition de supervision. Le chart est entièrement paramétrable : le registre d'images, l'étiquette de version, le nom d'hôte ou l'activation de certaines fonctionnalités se règlent par de simples valeurs, sans modifier le code du chart.

Ce packaging répond à un objectif de reproductibilité et de portabilité : le même chart, avec des valeurs différentes, peut déployer l'application en local, en pré-production ou en production. J'ai veillé à ce que le déploiement soit idempotent et à ce que la mise à jour d'une version se fasse par remplacement progressif des conteneurs, sans interruption de service perceptible par les utilisateurs.



Objets Kubernetes décrits et déployés par le chart Helm de l'application.

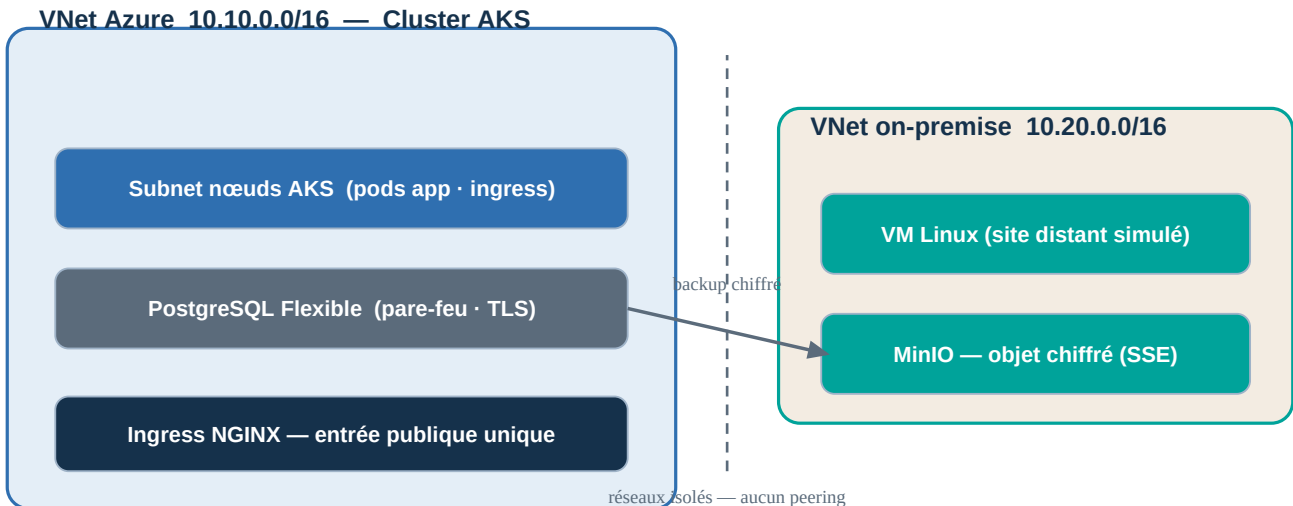
2.2 Résilience et autoscaling

J'ai configuré des sondes de vivacité et de disponibilité qui permettent à Kubernetes de redémarrer automatiquement un conteneur défaillant et de ne router le trafic que vers les instances prêtes à le recevoir. J'ai mis en place un autoscaler horizontal qui ajuste le nombre de répliques du backend en fonction de la charge processeur, dans une limite haute stricte afin de préserver la maîtrise des coûts — la scalabilité ne devant jamais se transformer en dérive budgétaire.

Cette combinaison assure la continuité de service, c'est-à-dire le volet PCA du projet. Nous avons vérifié en conditions réelles que la suppression manuelle d'une instance entraîne sa recreation automatique en quelques secondes, sans aucune intervention. J'ai par ailleurs défini le contexte de sécurité des conteneurs et les politiques de cloisonnement réseau, en concertation avec le responsable sécurité.

2.3 Le site on-premise simulé

Conformément au cahier des charges, j'ai provisionné une machine virtuelle représentant un site on-premise. Point essentiel : cette machine est placée dans un **réseau totalement séparé** de celui du cluster, sans interconnexion directe, ce qui la fait se comporter comme un site distant joint depuis le cloud — à l'image de ce que serait un véritable site relié par Internet ou par un lien sécurisé. Le schéma ci-dessous illustre cette isolation réseau volontaire :



Deux réseaux virtuels distincts, sans peering : le cloud héberge le cluster et la base, l'on-premise héberge MinIO ; seuls les flux de sauvegarde chiffrée les relient.

Ce choix de simulation nous a permis d'illustrer concrètement le principe d'une architecture hybride sans disposer d'un centre de données physique, tout en gardant la maîtrise des coûts : la machine peut être désactivée par une simple variable lorsque ce volet n'est pas démontré, ce qui évite toute dépense superflue en dehors des périodes d'utilisation.

2.4 Stockage objet chiffré (MinIO)

Sur cette machine, j'ai installé MinIO, un stockage objet compatible avec le standard S3, au moyen de playbooks Ansible. J'ai activé le chiffrement côté serveur, de sorte que les sauvegardes soient chiffrées au repos, répondant ainsi à l'exigence de protection des données sensibles formulée dans le cahier des charges. Un espace de stockage dédié, créé automatiquement, reçoit l'ensemble des sauvegardes de la base.

2.5 Automatisation par Ansible et plan de reprise

J'ai écrit les playbooks Ansible qui configurent la machine on-premise et automatisent les opérations de sauvegarde et de restauration. La sauvegarde réalise un export complet de la base et le dépose, chiffré, dans le stockage MinIO ; la restauration récupère la sauvegarde la plus récente et la réinjecte dans la base. Ansible assure ici la gestion de configuration, en complément de Terraform qui gère le provisionnement — une répartition classique et saine des responsabilités entre les deux outils.

J'ai rendu la restauration **idempotente**, c'est-à-dire rejouable même sur une base déjà peuplée, en incluant dans l'export les instructions de remise à zéro nécessaires. C'est cette propriété qui rend le plan de reprise réellement fiable et non seulement théorique : nous l'avons éprouvé de bout en bout selon le cycle suivant.



3. Choix technologiques : pourquoi ces technologies plutôt que d'autres

Trois décisions ont structuré mon périmètre : le gestionnaire de déploiement Kubernetes, la solution de stockage des sauvegardes et l'outil de configuration du site distant. Je les justifie ci-dessous par comparaison directe avec leurs alternatives.

3.1 Déploiement Kubernetes : Helm plutôt que Kustomize ou kubectl brut

Critère	Helm (retenu)	Kustomize	kubectl (manifs bruts)
Paramétrage	Valeurs + templates puissants	Superpositions (overlays)	Aucun (statique)
Réutilisabilité	Chart versionné, portable	Bonne	Faible (copier-coller)
Retour arrière	Natif (helm rollback)	Manuel	Manuel
Multi-environnements	Un chart, N jeux de valeurs	Overlays par env.	Fichiers dupliqués

Helm s'est imposé comme le gestionnaire de paquets de référence pour Kubernetes : il décrit un ensemble d'objets de façon paramétrable et versionnée, et gère proprement mises à jour et retours arrière. Kustomize, plus simple, ne propose pas de véritable gestion de version ni de commande de retour arrière ; déployer des manifestes bruts avec kubectl aurait imposé de dupliquer les fichiers pour chaque environnement. La capacité de **rollback** natif de Helm, précieuse en exploitation, et son modèle « un chart, plusieurs jeux de valeurs » ont été décisifs.

3.2 Stockage des sauvegardes : MinIO plutôt qu'Azure Blob Storage

Critère	MinIO (retenu)	Azure Blob Storage
Localisation	On-premise (site maîtrisé)	Cloud Azure
Démonstration hybride	Illustre le cahier des charges	Resterait dans le cloud
Compatibilité S3	Native	API propre (SDK Azure)
Chiffrement au repos	SSE activé	Oui (géré par Azure)
Portabilité	Déployable partout	Lié à Azure

MinIO a été retenu pour le stockage des sauvegardes car le cahier des charges imposait une architecture hybride : les données sensibles devaient être répliquées sur un site maîtrisé par l'organisation, hors du cloud. Azure Blob Storage aurait maintenu les sauvegardes dans le cloud, manquant l'objectif de démonstration hybride. MinIO est léger, compatible avec l'écosystème S3 — donc interopérable — et facile à déployer sur une simple machine ; sa compatibilité S3 faciliterait en outre une migration ultérieure vers un stockage objet cloud si le besoin s'en faisait sentir.

3.3 Configuration du site distant : Ansible plutôt que des scripts ou cloud-init

Critère	Ansible (retenu)	Scripts shell ad hoc	cloud-init
Idempotence	Native	À coder manuellement	Limitée (au 1er démarrage)

Critère	Ansible (retenu)	Scripts shell ad hoc	cloud-init
Lisibilité	Déclaratif (YAML)	Impératif, dispersé	Déclaratif mais figé
Rejouabilité	À tout moment	Risquée	Au démarrage seulement
Réutilisation	Rôles / playbooks	Faible	Faible

Le recours à **Ansible** pour configurer la machine distante, plutôt qu'à des scripts ad hoc ou à cloud-init, garantit une configuration déclarative, idempotente, rejouable à tout moment et documentée. Des scripts shell auraient exigé de coder manuellement l'idempotence ; cloud-init ne s'exécute qu'au premier démarrage et n'aurait pas permis de rejouer la configuration ni les opérations de sauvegarde. La combinaison Terraform (provisionnement) et Ansible (configuration) est une pratique éprouvée qui sépare clairement la création des ressources de leur paramétrage.

4. Difficultés rencontrées et solutions apportées

La difficulté centrale a été d'orchestrer une réplication du cloud vers l'on-premise à la fois fiable et rejouable. La première version de la restauration échouait sur une base déjà peuplée, en raison de conflits sur les objets existants ; je l'ai corrigée en rendant l'export auto-suffisant, capable de réinitialiser les structures avant de les recréer. Ce détail, en apparence mineur, fait toute la différence entre un plan de reprise qui ne fonctionne qu'en démonstration et un plan réellement exploitable en conditions dégradées.

J'ai également dû fiabiliser la connexion automatisée à la machine distante : la configuration Ansible devait attendre que la machine soit effectivement prête à accepter les connexions avant de la configurer. En coordination avec Elyess, nous avons ajouté une temporisation d'attente qui garantit la robustesse de cette étape lorsqu'elle est exécutée automatiquement par la chaîne, sans supposer que la machine est immédiatement disponible.

Enfin, la compatibilité de l'export de base avec la version du serveur a nécessité d'utiliser un outil aligné sur la version de PostgreSQL déployée, afin d'éviter tout écart susceptible de rendre une sauvegarde inutilisable — un point de vigilance essentiel pour un plan de reprise digne de confiance.

5. Perspectives d'évolution

La première évolution consisterait à activer un module réseau appliquant réellement les politiques de cloisonnement préparées, afin d'obtenir une isolation effective entre les composants du cluster et de renforcer la sécurité interne.

Je viserais ensuite une haute disponibilité répartie sur plusieurs zones, de sorte que la panne d'une zone n'interrompe pas le service, ainsi qu'une réplication géographique des sauvegardes pour se prémunir contre la perte d'un site entier.

Je souhaiterais également planifier des tests de restauration réguliers et automatiques : un plan de reprise ne vaut que s'il est éprouvé périodiquement, faute de quoi l'on découvre ses failles au pire moment. Externaliser les sauvegardes vers un stockage redondant compléterait ce dispositif.

Enfin, remplacer le site on-premise simulé par une véritable intégration réseau, reliée par un lien sécurisé de type VPN, rapprocherait encore la maquette d'une architecture hybride de production et permettrait d'éprouver les problématiques réelles de latence et de sécurité des échanges.

6. Analyse critique des limites

Le site on-premise est ici simulé par une machine virtuelle placée dans un réseau distinct : l'isolation est représentative d'un site séparé, mais il ne s'agit pas d'un véritable centre de données physique, et l'interconnexion réelle par VPN n'a pas été mise en œuvre dans ce cadre.

De plus, la restauration ramène l'état de la base au moment de la dernière sauvegarde : la perte de données potentielle dépend donc directement de la fréquence des sauvegardes, ce que l'on désigne par l'objectif de point de reprise. Une fréquence plus élevée réduirait cette perte, au prix d'un stockage et d'un trafic accrus.

Enfin, le stockage MinIO fonctionne en instance unique, sans redondance interne, ce qui constituerait un point de défaillance en production ; et l'architecture réseau retenue pour le cluster, choisie pour sa légèreté, limite l'application stricte des politiques de cloisonnement que j'ai définies. Ces limites sont assumées au regard du caractère MVP du projet et clairement documentées.

7. Annexe — documentation utilisateur (mon périmètre)

7.1 Consulter le stockage de sauvegarde

La console web de MinIO, accessible sur la machine on-premise, permet de visualiser l'espace de stockage dédié et d'y retrouver les sauvegardes chiffrées, chacune horodatée. C'est le moyen le plus simple de vérifier qu'une sauvegarde a bien été produite et d'en connaître la date.

7.2 Déclencher une sauvegarde

Les sauvegardes sont planifiées quotidiennement de façon automatique, mais peuvent aussi être déclenchées à la demande depuis le workflow prévu à cet effet, en choisissant le mode de sauvegarde. L'opération réalise l'export chiffré de la base et son dépôt dans MinIO.

7.3 Restaurer les données

En cas de perte, la restauration se déclenche depuis le même workflow en choisissant le mode de restauration : la sauvegarde la plus récente est récupérée puis réinjectée dans la base. Grâce au caractère idempotent de l'opération, elle peut être relancée sans risque même si la base contient déjà des données.

7.4 Adapter le déploiement

Le chart Helm étant paramétrable, il est possible d'ajuster le registre d'images, le nom d'hôte d'accès, le nombre de répliques ou l'activation de certaines fonctionnalités par de simples valeurs, sans modifier le code du chart, ce qui facilite l'adaptation de l'application à un nouvel environnement.

8. Analyse personnelle

Défis rencontrés

Mon principal défi a été de concevoir une réplication du cloud vers l'on-premise fiable et rejouable, et de rendre la restauration réellement idempotente. Il m'a fallu comprendre en profondeur le comportement de l'outil de sauvegarde pour transformer une procédure fragile en un plan de reprise digne de confiance.

Le caractère hybride du sujet, à cheval entre le cluster cloud et une machine distante, a également représenté un défi d'intégration : il a fallu faire dialoguer proprement deux environnements conçus pour être séparés.

Forces

Je retiens ma maîtrise du packaging Kubernetes et des objets qui le composent, ainsi qu'une vision claire des enjeux de résilience et d'architecture hybride.

J'ai su relier des sujets souvent traités séparément — orchestration, réseau, sauvegarde — pour en faire un ensemble cohérent, ce qui témoigne d'une capacité à prendre de la hauteur sur l'architecture globale.

Faiblesses

Je dois passer d'un on-premise simulé à une intégration réseau plus réaliste, ce qui suppose d'approfondir mes compétences en interconnexion réseau, notamment les VPN et le routage entre sites.

Par ailleurs, la redondance du stockage de sauvegarde reste à renforcer : une instance unique, bien que suffisante pour une démonstration, ne serait pas acceptable pour un usage réel.

Compétences développées

Ce projet a consolidé mes compétences sur Kubernetes et Helm — templating, autoscaling, rôles, politiques réseau — sur Ansible, et sur la conception d'un plan de reprise d'activité complet, de la sauvegarde chiffrée jusqu'à la restauration automatisée.

J'ai aussi appris à raisonner en termes d'objectifs de reprise — délai et point de reprise — qui structurent toute réflexion sérieuse sur la continuité d'activité.

Axes d'amélioration personnels

Je souhaite mettre en œuvre une haute disponibilité multi-zones et automatiser des tests de restauration réguliers, afin de garantir la validité du plan de reprise dans la durée et non seulement à un instant donné.

Je compte enfin me former à l'interconnexion sécurisée de sites, pour être capable de concevoir de véritables architectures hybrides reliant un centre de données à un cloud public.